

Artificial Intelligence

How the Amazon.com Catalog Team built self-learning generative AI at scale with Amazon Bedrock

by Tarik Arici, Amin Banitalebi, Erdinc Basci, Mey Meenakshisundaram, Puneet Sahni, and Sameer Thombare | on 23 JAN 2026 | in [Amazon Bedrock](#), [Amazon Bedrock AgentCore](#), [Customer Solutions](#), [Generative AI](#), [Technical How-to](#) | [Permalink](#) | [Comments](#) | [Share](#)

The [Amazon.com](#) Catalog is the foundation of every customer's shopping experience—the definitive source of product information with attributes that power search, recommendations, and discovery. When a seller lists a new product, the catalog system must extract structured attributes—dimensions, materials, compatibility, and technical specifications—while generating content such as titles that match how customers search. A title isn't a simple enumeration like color or size; it must balance seller intent, customer search behavior, and discoverability. This complexity, multiplied by millions of daily submissions, makes catalog enrichment an ideal proving ground for self-learning AI.

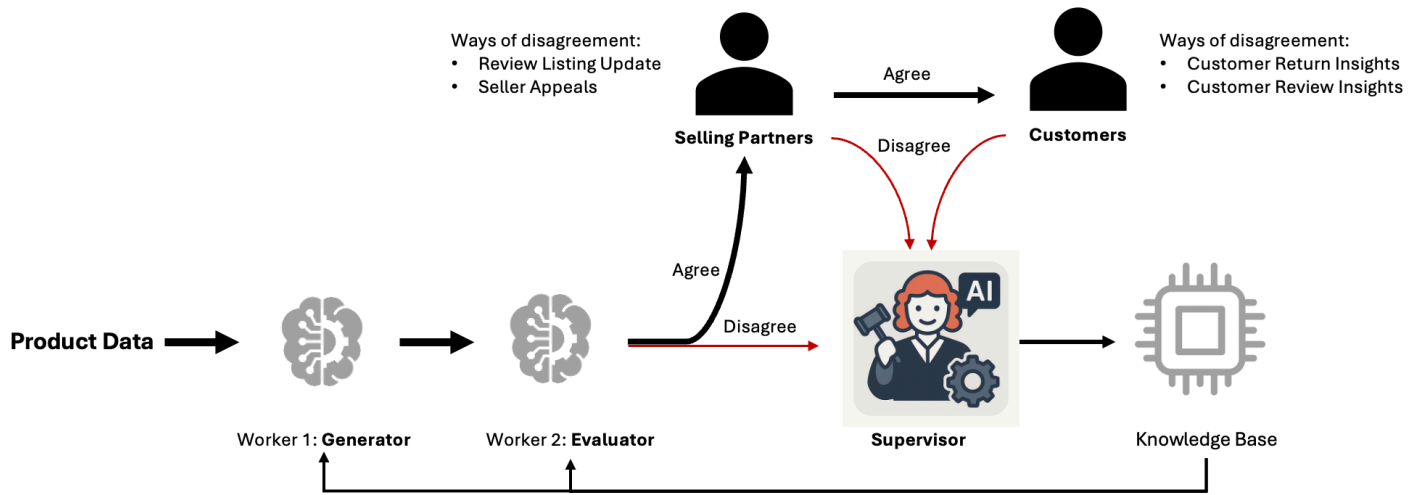
In this post, we demonstrate how the Amazon Catalog Team built a self-learning system that continuously improves accuracy while reducing costs at scale using [Amazon Bedrock](#).

The challenge

In generative AI deployment environments, improving model performance calls for constant attention. Because models process millions of products, they inevitably encounter edge cases, evolving terminology, and domain-specific patterns where accuracy may degrade. The traditional approach—applied scientists analyzing failures, updating prompts, testing changes, and redeploying—works but is resource-intensive and struggles to keep pace with real-world volume and variety. The challenge isn't whether we can improve these systems, but how to make improvement scalable and automatic rather than dependent on manual intervention. At Amazon Catalog, we faced this challenge head-on. The tradeoffs seemed impossible: large models would deliver accuracy but wouldn't scale efficiently to our volume, while smaller models struggled with the complex, ambiguous cases where sellers needed the most help.

Solution overview

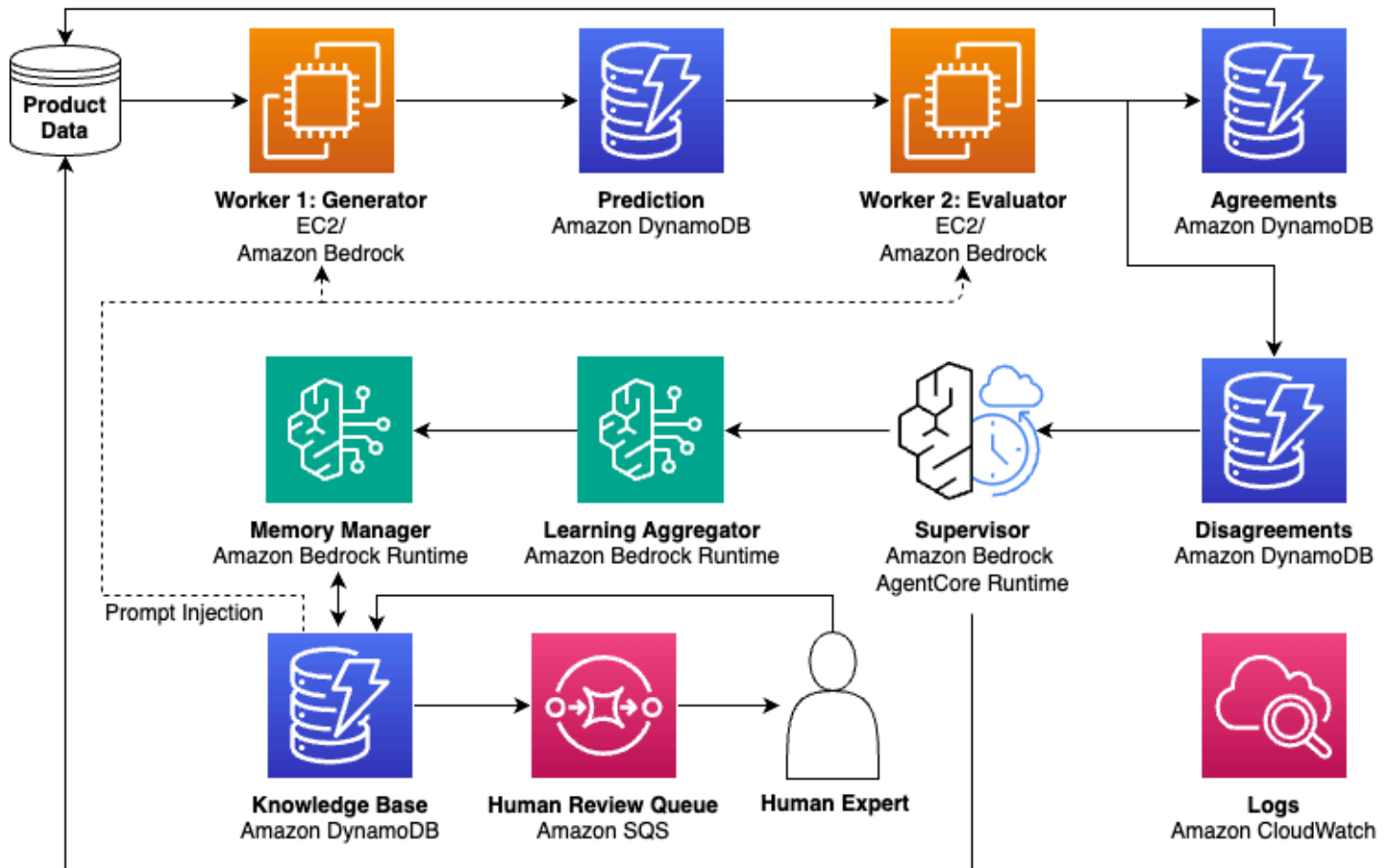
Our breakthrough came from an unconventional experiment. Instead of choosing a single model, we deployed multiple smaller models to process the same products. When these models agreed on an attribute extraction, we could trust the result. But when they disagreed—whether from genuine ambiguity, missing context, or one model making an error—we discovered something profound. These disagreements weren't always errors, but they were almost always indicators of complexity. This led us to design a self-learning system that reimagines how generative AI scales. Multiple smaller models process routine cases through consensus, invoking larger models only when disagreements occur. The larger model is implemented as a supervisor agent with access to specialized tools for deeper investigation and analysis. But the supervisor doesn't just resolve disputes; it generates reusable learnings stored in a dynamic knowledge base that helps prevent entire classes of future disagreements. We invoke more powerful models only when the system detects high learning value at inference time, while correcting the output. The result is a self-learning system where costs decrease and quality increases—because the system learns to handle edge cases that previously triggered supervisor calls. Error rates fell continuously, not through retraining but through accumulated learnings from resolved disagreements injected into smaller model prompts. The following figure shows the architecture of this self-learning system.



In the self-learning architecture, product data flows through generator-evaluator workers, with disagreements routed to a supervisor for investigation. Post-inference, the system also captures feedback signals from sellers (such as listing updates and appeals) and customers (such as returns and negative reviews). Learnings from the sources are stored in a hierarchical knowledge base and injected back into worker prompts, creating a continuous improvement loop.

The following describes a simplified reference architecture that demonstrates how this self-learning pattern can be implemented using AWS services. While our production system has additional complexity, this example illustrates the core components and data flows.

This system can be built with Amazon Bedrock, which provides the essential infrastructure for multi-model architectures. The ability of Amazon Bedrock to access diverse foundation models enables teams to deploy smaller, efficient models like Amazon Nova Lite as workers and more capable models like Anthropic Claude Sonnet as supervisors—optimizing both cost and performance. For even greater cost efficiency at scale, teams can also deploy open source small models on [Amazon Elastic Compute Cloud \(Amazon EC2\)](#) GPU instances, providing full control over worker model selection and batch throughput optimization. For productionizing a supervisor agent with its specialized tools and dynamic knowledge base, Bedrock AgentCore provides the runtime scalability, memory management, and observability needed to deploy self-learning systems reliably at scale.



Our supervisor agent integrates with Amazon's extensive Selection and Catalog Systems. The above diagram is a simplified view showing the key features of the agent and some of the AWS services that make it possible. Product data flows through generator-evaluator workers (Amazon EC2 and [Amazon Bedrock Runtime](#)), with agreements stored directly and disagreements routed to a supervisor agent (Bedrock AgentCore). The learning aggregator and memory manager utilize [Amazon DynamoDB](#) for the knowledge base, with learnings injected back into worker prompts. Human review ([Amazon Simple Queue Service \(Amazon SQS\)](#)) and observability ([Amazon CloudWatch](#)) complete the architecture. Production implementations will likely require additional components for scale, reliability, and integration with existing systems.

But how did we arrive at this architecture? The key insight came from an unexpected place.

The insight: Turning disagreements into opportunities

Our perspective shifted during a debugging session. When multiple smaller models (such as Nova Lite) disagreed on product attributes—interpreting the same specification differently based on how they understood technical terminology—we initially saw this as a failure. But the data told a different story: products where our smaller models disagreed correlated with cases requiring more manual review and clarification. When models disagreed, those were precisely the products that needed additional investigation. The disagreements were surfacing learning opportunities, but we couldn't have engineers and scientists deep-dive on every case. The supervisor agent does this automatically at scale. And crucially, the goal isn't just to determine which model was right—it's to extract learnings that help prevent similar disagreements in the future. This is the key to efficient scaling. Disagreements don't just come from AI workers at inference time. Post-inference, sellers express disagreement through listing updates and appeals—signals that our original extraction might have missed important context. Customers disagree through returns and negative reviews, often indicating that product information didn't

match expectations. These post-inference human signals feed into the same learning pipeline, with the supervisor investigating patterns and generating learnings that help prevent similar issues across future products. We found a sweet spot: attributes with moderate AI worker disagreement rates yielded the richest learnings—high enough to surface meaningful patterns, low enough to indicate solvable ambiguity. When disagreement rates are too low, they typically reflect noise or fundamental model limitations rather than learnable patterns—for those, we consider using more capable workers. When disagreement rates are too high, it signals that worker models or prompts aren't yet mature enough, triggering excessive supervisor calls that undermine the efficiency gains of the architecture. These thresholds will vary by task and domain; the key is identifying your own sweet spot where disagreements represent genuine complexity worth investigating, rather than fundamental gaps in worker capability or random noise.

Deep dive: How it works

At the heart of our system are multiple lightweight worker models operating in parallel—some as generators extracting attributes, others as evaluators assessing those extractions. These workers can be implemented in a non-agentic way with fixed inputs, making them batch-friendly and scalable. The generator-evaluator pattern creates productive tension, conceptually similar to the productive tension in [generative adversarial networks](#) (GANs), though our approach operates at inference time through prompting rather than training. We explicitly prompt evaluators to be critical, instructing them to scrutinize extractions for ambiguities, missing context, or potential misinterpretations. This adversarial dynamic surfaces disagreements that represent genuine complexity rather than letting ambiguous cases pass through undetected. When the generator and evaluator agree, we have high confidence in the result and process it at minimal computational cost. This consensus path handles most product attributes. When they disagree, we've identified a case worth investigating—triggering the supervisor to resolve the dispute and extract reusable learnings.

Our architecture treats disagreement as a universal learning signal. At inference time, worker-to-worker disagreements catch ambiguity. Post-inference, seller feedback catches misalignments with intent and customer feedback catches misalignments with expectations. The three channels feed the supervisor, which extracts learnings that improve accuracy across the board. When workers disagree, we invoke a supervisor agent—a more capable model that resolves the dispute and investigates why it occurred. The supervisor determines what context or reasoning the workers lacked, and these insights become reusable learnings for future cases. For example, when workers disagreed about usage classification for a product based on certain technical terms, the supervisor investigated and clarified that those terms alone were insufficient—visual context and other indicators needed to be considered together. The supervisor generated a learning about how to properly weight different signals for that product category. This learning immediately updated our knowledge base, and when injected into worker prompts for similar products, helped prevent future disagreements across thousands of items. While the workers could theoretically be the same model as the supervisor, using smaller models is crucial for efficiency at scale. The architectural advantage emerges from this asymmetry: lightweight workers handle routine cases through consensus, while the more capable supervisor is invoked only when disagreements surface high-value learning opportunities. As the system accumulates learnings and disagreement rates drop, supervisor calls naturally decline—efficiency gains are baked directly into the architecture. This worker-supervisor heterogeneity also enables richer investigation. Because supervisors are invoked selectively, they can afford to pull in additional signals—customer reviews, return reasons, seller history—that would be impractical to retrieve for every product but provide crucial context when resolving complex disagreements. When these signals yield generalizable insights about how customers want product information presented—which attributes to highlight, what terminology resonates, how to frame specifications—the resulting learnings benefit future inferences across

similar products without retrieving those resource-intensive signals again. Over time, this creates a feedback loop: better product information leads to fewer returns and negative reviews, which in turn reflects improved customer satisfaction.

The knowledge base: Making learnings scalable

The supervisor investigates disagreements at the individual product level. With millions of items to process, we need a scalable way to transform these product-specific insights into reusable learnings. Our aggregation strategy adapts to context: high-volume patterns get synthesized into broader learnings, while unique or critical cases are preserved individually. We use a hierarchical structure where a large language model (LLM)-based memory manager navigates the knowledge tree to place each learning. Starting from the root, it traverses categories and subcategories, deciding at each level whether to continue down an existing path, create a new branch, merge with existing knowledge, or replace outdated information. This dynamic organization allows the knowledge base to evolve with emerging patterns while maintaining logical structure. During inference, workers receive relevant learnings in their prompts based on product category, automatically incorporating domain knowledge from past disagreements. The knowledge base also introduces traceability—when an extraction seems incorrect, we can pinpoint exactly which learning influenced it. This shifts auditing from an unscalable task to a practical one: instead of reviewing a sample of millions of outputs—where human effort grows proportionally with scale—teams can audit the knowledge base itself, which remains relatively fixed in size regardless of inference volume. Domain experts can directly contribute by adding or refining entries, no retraining required. A single well-crafted learning can immediately improve accuracy across thousands of products. The knowledge base bridges human expertise and AI capability, where automated learnings and human insights work together.

Lessons learned and best practices

When this self-learning architecture works best:

- **High-volume inference** where input diversity drives compounded learning
- **Quality-critical applications** where consensus provides natural quality assurance
- **Evolving domains** with new patterns and terminology constantly emerging

It's less suitable for low-volume scenarios (insufficient disagreements for learning) or use cases with fixed, unchanging rules.

Critical success factors:

- **Defining disagreements:** With a generator-evaluator pair, disagreement occurs when the evaluator flags the extraction as needing improvement. With multiple workers, scale thresholds accordingly. The key is maintaining productive tension between workers. If disagreement rates fall outside the productive range (too low or too high), consider more capable workers or refined prompts.
- **Tracking learning effectiveness:** Disagreement rates must decrease over time—this is your primary health metric. If rates stay flat, check knowledge retrieval, prompt injection, or evaluator criticality.
- **Knowledge organization:** Structure learnings hierarchically and keep them actionable. Abstract guidance doesn't help; specific, concrete learnings directly improve future inferences.

Common pitfalls

- **Focusing on cost over intelligence:** Cost reduction is a byproduct, not the goal
- **Rubber-stamp evaluators:** Evaluators that simply approve generator outputs won't surface meaningful disagreements—prompt them to actively challenge and critique extractions
- **Poor learning extraction:** Supervisors must identify generalizable patterns, not just fix individual cases
- **Knowledge rot:** Without organization, learnings become unsearchable and unusable

The key insight: treat declining disagreement rates as your north star metric—they show the system is truly learning.

Deployment strategies: Two approaches

- **Learn-then-deploy:** Start with basic prompts and let the system learn aggressively in a pre-production environment. Domain experts then audit the knowledge base—not individual outputs—to make sure learned patterns align with desired outcomes. When approved, deploy with validated learnings. This is ideal for new use cases where you don't yet know what *good* looks like—disagreements help discover the right patterns, and knowledge base auditing lets you shape them before production.
- **Deploy-and-learn:** Start with refined prompts and good initial quality, then continuously improve through ongoing learning in production. This works best for well-understood use cases where you can define quality upfront but still want to capture domain-specific nuances over time.

Both approaches use the same architecture—the choice depends on whether you're exploring new territory or optimizing familiar ground.

Conclusion

What started as an experiment in catalog enrichment revealed a fundamental truth: AI systems don't have to be frozen in time. By embracing disagreements as learning signals rather than failures, we've built an architecture that accumulates domain knowledge through actual usage. We watched the system evolve from generic understanding to domain-specific expertise. It learned industry-specific terminology. It discovered contextual rules that vary across categories. It adapted to requirements no pre-trained model would encounter—all without retraining, through learnings stored in a knowledge base and injected back into worker prompts. For teams operationalizing similar architectures, Amazon Bedrock AgentCore offers purpose-built capabilities:

- [AgentCore Runtime](#) handles quick consensus decisions for routine cases while supporting extended reasoning when supervisors investigate complex disagreements
- [AgentCore Observability](#) provides visibility into which learnings drive impact, helping teams refine knowledge propagation and maintain reliability at scale

The implications extend beyond catalog management. High-volume AI applications could benefit from this process—and the ability of Amazon Bedrock to access diverse models makes this architecture straightforward to implement. The key insight is this: we've shifted from asking "which model should we use?" to "how can we build systems that learn our specific patterns?" Whether you learn-then-deploy for new use cases or deploy-and-learn for established ones, the implementation is straightforward: start with workers suited to your task, choose a supervisor, and let disagreements drive learning. With the right architecture, every inference can become an opportunity to capture domain knowledge. That's not just scaling—that's building institutional knowledge into your AI systems.

Acknowledgement

This work wouldn't have been possible without the contributions and support from **Ankur Datta** (Senior Principal Applied Scientist – leader of science in Everyday Essentials Stores), **Zhu Cheng** (Applied Scientist), **Xuan Tang** (Software Engineer), **Mohammad Ghasemi** (Applied Scientist). We sincerely appreciate the contributions in designs, implementations, numerous fruitful brain-storming sessions, and all the insightful ideas and suggestions.

About the authors



Tarik Arici is a Principal Scientist at Amazon Selection and Catalog Systems (ASCS), where he pioneers self-learning generative AI systems design for catalog quality enhancement at scale. His work focuses on building AI systems that automatically accumulate domain knowledge through production usage—learning from customer reviews and returns, seller feedback, and model disagreements to improve quality while reducing costs. Tarik holds a PhD in Electrical and Computer Engineering from Georgia Institute of Technology.



Sameer Thombare is a Senior Product Manager at Amazon with over a decade of experience in Product Management, Category/P&L Management across diverse industries, including heavy engineering, telecommunications, finance, and eCommerce. Sameer is passionate about developing continuously improving closed-loop systems and leads strategic initiatives within Amazon Selection and Catalog Systems (ASCS) to build a sophisticated self-learning closed-loop system that synthesize signals from customers, sellers, and supply chain operations to optimize outcomes. Sameer holds an MBA from the Indian Institute of Management Bangalore and an engineering degree from Mumbai University.



Amin Banitalebi received his PhD in the Digital Media at the University of British Columbia (UBC), Canada, in 2014. Since then, he has taken various applied science roles spanning over areas in computer vision, natural language processing, recommendation systems, classical machine learning, and generative AI. Amin has co-authored over 90 publications and patents. He is currently an Applied Science Manager in Amazon Everyday Essentials.



Puneet Sahni is a Senior Principal Engineer at Amazon Selection and Catalog Systems (ASCS), where he has spent over 8 years improving the completeness, consistency, and correctness of catalog data. He specializes in catalog data modeling and its application to enhancing Selling Partner and customer experiences, while using ML/DL and LLM-based enrichment to drive improvements in catalog data quality.



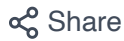
Erdinc Basci joined Amazon in 2015 and brings over 23 years of technology industry experience. At Amazon, he has led the evolution of Catalog system architectures—including ingestion pipelines, prioritized processing, and traffic shaping—as well as catalog data architecture improvements such as segmented offers, product specifications for manufacture-on-demand products, and catalog data experimentation. Erdinc has championed a hands-on performance engineering culture across Amazon services unlocking \$1B+ annualized cost savings and 20%+ latency wins across core Stores services. He is currently focused on

improving generative AI application performance and GPU efficiency across Amazon. Erdinc holds a BS in Computer Science from Bilkent University, Turkey, and an MBA from Seattle University, US.



Mey Meenakshisundaram is a Director in Amazon Selection and Catalog Systems, where he leads innovative GenAI solutions to establish Amazon's worldwide catalog as the best-in-class source for product information. His team pioneers advanced machine learning techniques, including multi-agent systems and large language models, to automatically enrich product attributes and improve catalog quality at scale. High-quality product information in the catalog is critical for delighting customers in finding the right products, empowering selling partners to list their products effectively, and enabling Amazon operations to reduce manual

effort.



Comments

[Log in to comment](#)